CS313X • INFORMATION RETRIEVAL

Week 5: Boolean & Vector Space Models

From Exact Matching to Ranked Retrieval

Course Plan: Week 5

1. **Boolean Retrieval**
   Exact matching with AND, OR, NOT logic.

2. **Scoring & Ranking**
   Moving beyond binary match to relevance scores.

3. **Term Weighting (TF-IDF)**
   Why rare terms matter more than common ones.

4. **Vector Space Model**
   Cosine similarity and document vectors.

5. **Real-World Applications**
   E-commerce search, news recommendations, and efficiency.

Boolean Retrieval Model

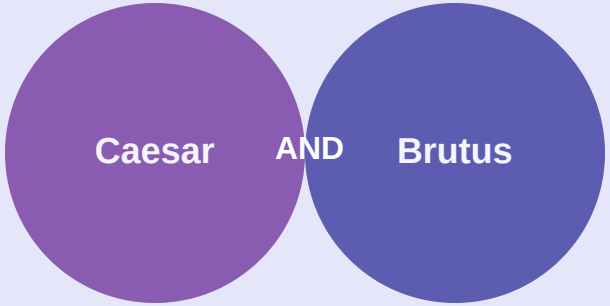
What is it?

The simplest retrieval model based on **Set Theory** and **Boolean Logic**. Documents either match the query exactly or they don't.

Key Characteristics:

- **Exact Match:** Binary outcome (1 or 0).
- **Precise Control:** User defines exact criteria.
- **Used In:** Legal, Medical, and Library systems.

LOGIC VISUALIZATION

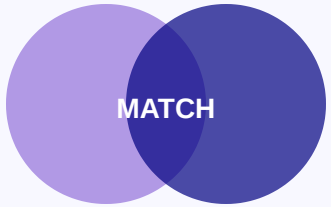



```
Caesar AND Brutus NOT Calpurnia
```

Boolean Logic Operations

AND

Intersection



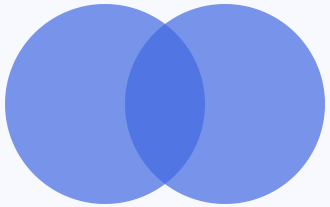
Retrieves documents containing **BOTH** terms.

Reduces result set size (Restrictive).

`Brutus AND Caesar`

OR

Union



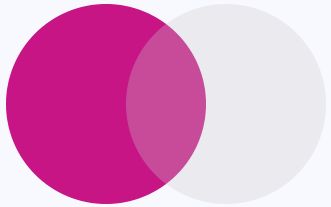
Retrieves documents containing **EITHER** term (or both).

Increases result set size (Inclusive).

`Brutus OR Caesar`

NOT

Difference



Retrieves documents containing the first term but **EXCLUDES** the second.

Used to filter out noise.

`Brutus NOT Caesar`

Recap: Inverted Index

> The core data structure for Boolean retrieval. It maps **terms** to the **documents** that contain them.

`Brutus`	→	1, 2, 4, 11, 31	*(DF = 5)*
`Caesar`	→	1, 2, 4, 5, 6, 16	*(DF = 6)*
`Calpurnia`	→	2, 31, 54, 101	*(DF = 4)*

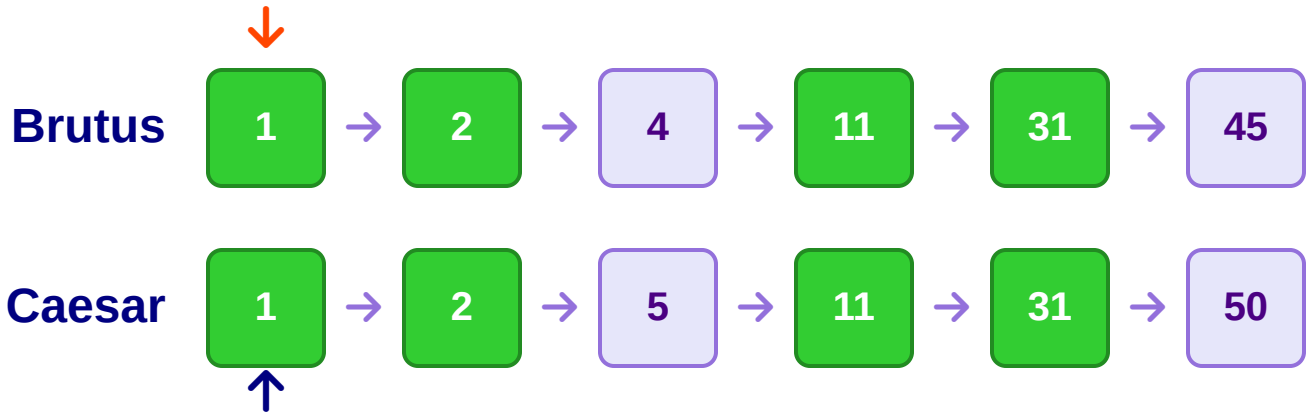
DocIDs are always **sorted**.

Sparse storage (only non-zero entries).

Optimized for **intersection**.

Processing Boolean Queries

Brutus AND Caesar



The Intersection Algorithm (Merge)

1. Locate **Brutus** and **Caesar** in the Dictionary.
2. Retrieve their postings lists.
3. Walk through both lists simultaneously using pointers.
4. If DocIDs match, add to result. If not, advance the pointer of the smaller DocID.

Result: [1, 2, 11, 31]

Query Optimization

Goal: Process queries faster without changing the result.
Key Insight: The order of operations matters significantly!

NAIVE APPROACH

```
(Brutus AND Caesar) AND Calpurnia
      50k       100k          5
```

1. Brutus AND Caesar **~50,000 ops**
2. Result AND Calpurnia **~50,000 ops**

Total: ~100,000 Ops

OPTIMIZED APPROACH

```
(Calpurnia AND Brutus) AND Caesar
      5          50k          100k
```

1. Calpurnia AND Brutus **~5 ops**
2. Result AND Caesar **~5 ops**

Total: ~10 Ops

💡 **Rule: Process the smallest postings lists first!**

Optimization Strategies

The Golden Rule of Intersection

Process terms in order of **increasing document frequency**. Start with the rarest term first!

NAIVE ORDER

```
(Brutus AND Caesar) AND Calpurnia
50k docs    60k docs      2 docs
```

- Merge Brutus (50k) & Caesar (60k)
- → Intermediate Result: ~30k docs
- Merge Result (30k) & Calpurnia (2)

Total Ops: ~140,000

OPTIMIZED ORDER

```
(Calpurnia AND Brutus) AND Caesar
2 docs      50k docs    60k docs
```

- Merge Calpurnia (2) & Brutus (50k)
- → Intermediate Result: <= 2 docs
- Merge Result (2) & Caesar (60k)

Total Ops: ~110,000

Limitations of Boolean Retrieval

Exact Match Rigidity

Documents either match or they don't. There is no "partial match" or "somewhat relevant."

Binary: 0 or 1

"Feast or Famine"

Queries often return **10,000+** documents (too broad) or **0** documents (too specific).

Hard to Tune

Lack of Ranking

Result set is unordered. A document mentioning the term once is equal to one mentioning it 100 times.

All Results Equal

Introduction to Ranked Retrieval

Boolean Retrieval

- **Feast or Famine:**
  Returns either too many results (10,000+) or zero.
- **No Nuance:**
  A document with 1 term is treated same as one with 100.
- **Complex Queries:**
  Requires users to know AND, OR, NOT.

Ranked Retrieval

- **Relevance Scoring:**
  Assigns a score (0.0 to 1.0) to each document.
- **Top K Results:**
  User only sees the "best" 10 documents.
- **Free Text Queries:**
  "President Lincoln" works naturally.

The Concept of Scoring

The Goal: Instead of just saying "Yes" or "No", we want to assign a **score** to each document that measures how well it matches the query.

BOOLEAN RETRIEVAL

Match / No Match

Unsorted Set
(Too many or too few)

RANKED RETRIEVAL

1	Doc 142	0.95
2	Doc 89	0.80
3	Doc 33	0.60
4	Doc 12	0.40

$$Score(q, d) \in [0, 1]$$

Jaccard Coefficient

A simple statistic used for comparing the **similarity** and **diversity** of sample sets. It measures the overlap between two sets relative to their total size.

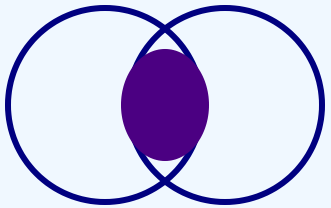
THE FORMULA

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}$$

Range: 0 (No overlap) to 1 (Identical)

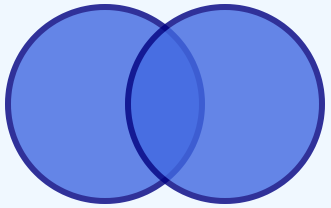
Intersection

(Numerator)



Union

(Denominator)



Jaccard Coefficient: Example

QUERY (A)

"ides of march"

```
A = {ides, of, march}
```

DOCUMENT (B)

"caesar died in march"

```
B = {caesar, died, in, march}
```

Step 1: Intersection

A ∩ B	{march}
Size	1

Step 2: Union

A ∪ B	{ides, of, march, caesar, died, in}
Size	6

J(A,B) = 1 / 6 ≈ 0.167

Limitations of Jaccard Coefficient

⚠ Ignores Term Frequency

Jaccard only considers the **presence** (1) or **absence** (0) of terms. It does not care how many times a term appears.

Doc A:	`"apple"`
Doc B:	`"apple apple apple"`
Set View:	**`{apple} == {apple}`**

Length Bias

The denominator **|A ∪ B|** penalizes long documents. A relevant document that is very long (has many unique words) will have a large Union, resulting in a **lower score** compared to a short document.

Conclusion: We need a model that accounts for Frequency.

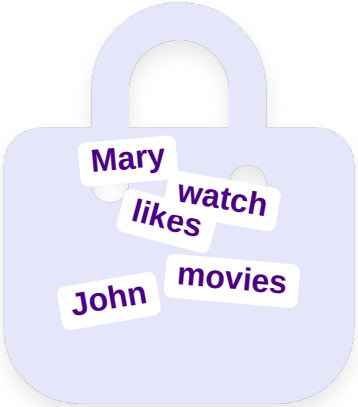
The Bag of Words (BoW) Model

A simplified representation used in Information Retrieval where text is represented as a **multiset** of its words, disregarding grammar and word order but keeping **multiplicity**.

RAW TEXT

```
"John likes to watch movies. Mary likes movies too."
```

THE "BAG"



TERM COUNTS

"likes"	2
"movies"	2
"John"	1
"Mary"	1
"watch"	1

⚠ **Key Limitation: "John loves Mary" == "Mary loves John"**

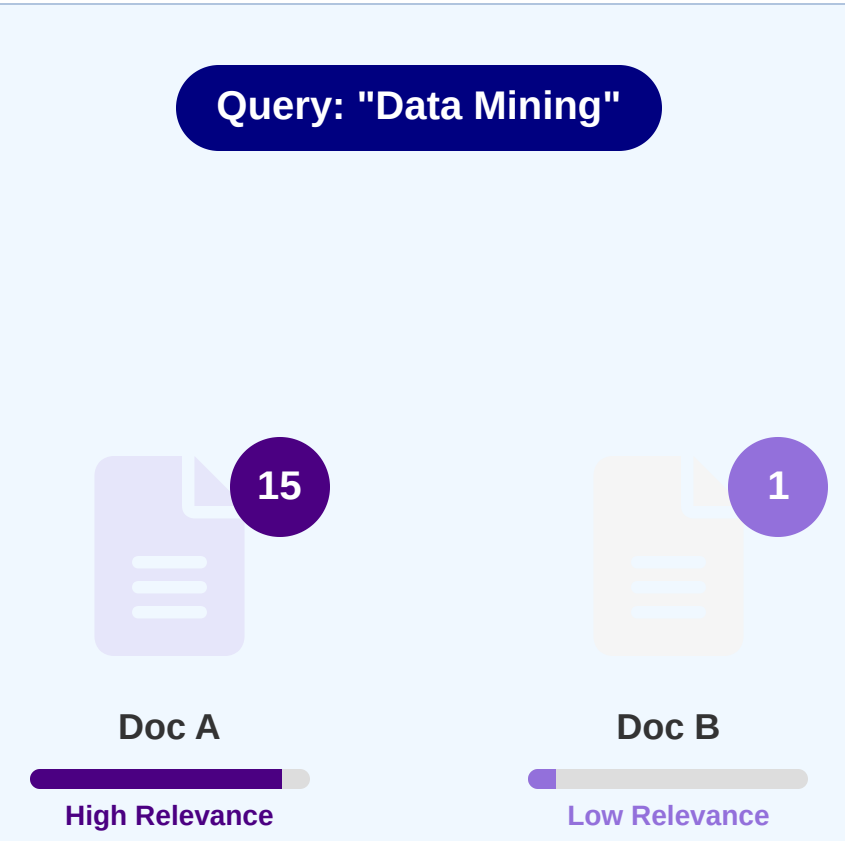
Term Frequency (TF)

The Assumption: The more often a term *t* occurs in a document *d*, the more likely it is that *d* is relevant to the topic *t*.

$$tf_{t,d} = count(t, d)$$

Bag of Words Model:
We ignore the *order* of words. We only care about their *counts*.

Raw Frequency:
The simplest weight is just the number of occurrences.



"Doc A mentions 'mining' 15 times, so it's likely the main topic."

Log-Frequency Weighting

⚠ The Problem with Raw TF

Relevance does **not** increase linearly with term frequency. A document with 10 occurrences is more relevant than one with 1, but not 10 times more relevant.

EXAMPLE

Query: "autism"

Doc A: mentions "autism" 1 time.
Doc B: mentions "autism" 10 times.

Doc B is more relevant, but is it 10x better?
Likely not. We need to **dampen** the effect.

$$w_{t,d} = 1 + \log_{10}(tf_{t,d})$$

if $tf_{t,d} > 0$, else 0

Raw TF	Log Weight (w)
0	0
1	1
10	2
100	3
1000	4

TF Calculation Example

Scenario:
A document contains the term "apple" **100 times**.

Formula: $w = 1 + \log_{10}(tf)$

Input: $tf = 100$

Calc: $w = 1 + \log_{10}(100)$

Step: $w = 1 + 2$

Weight = 3

THE DAMPENING EFFECT

Raw Count (tf)		Log Weight (w)
0	→	0
1	→	1
2	→	1.3
10	→	2
1,000	→	4
1,000,000	→	**7**

Key Insight:
A document with 1 million occurrences is only **7 times** more relevant than a document with 1 occurrence, not 1 million times.

Document Frequency (DF)

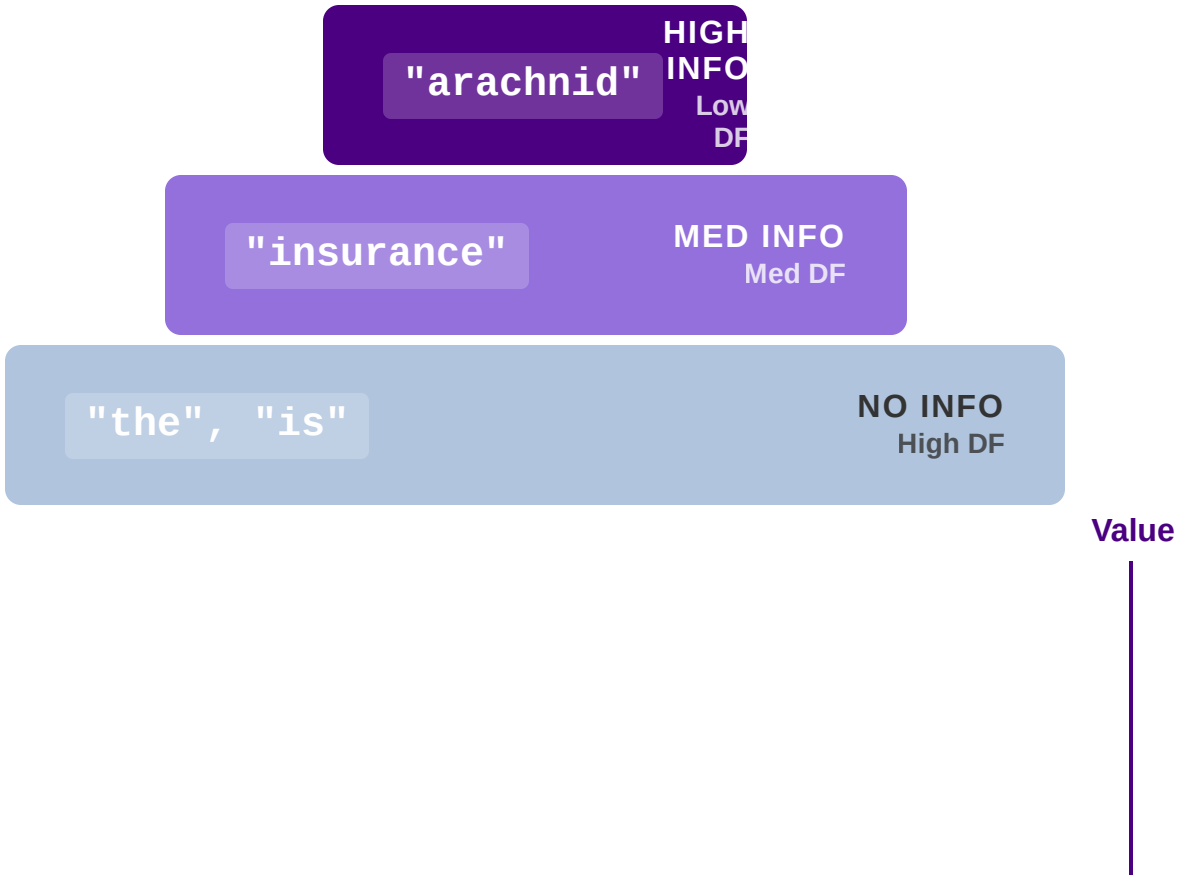
DEFINITION

$\mathbf{df_t}$ is the number of documents in the collection that contain the term *t*.

THE LOGIC

Rare terms are more **informative** than frequent terms. If a term appears in every document, it tells us nothing about relevance.

Goal: Assign higher weights to rare terms (low DF) and lower weights to common terms (high DF).



Inverse Document Frequency (IDF)

The Intuition: Rare terms are informative. Frequent terms are noise.
We want a weight that is **high** when the document frequency (df) is **low**.

$$\mathrm{idf}_t = \log_{10} \frac{N}{\mathrm{df}_t}$$

N = Total number of documents in collection

df_t = Number of documents containing term *t*

FREQUENT (LOW VALUE) — RARE (HIGH VALUE)

"the"	*"computer"*	*"arachnocampa"*
idf ≈ 0	idf ≈ 2.5	idf ≈ 6.0

IDF Calculation

$$idf_t = \log_{10}(N / df_t)$$

N = Total number of documents

df_t = Document frequency of term t

Scenario: Collection size N = 1,000,000

Term (t)	Document Frequency (df_t)	N / df_t	IDF ($\log_{10}$)
"the"	1,000,000	1	**0**
"animal"	100,000	10	**1**
"sunday"	1,000	1,000	**3**
"pentagon"	10	100,000	**5**

TF-IDF Weighting

The best known weighting scheme in information retrieval. It combines two forces: **local frequency** (TF) and **global rarity** (IDF).

$$(1 + \log(tf)) \times \log(N / df) = w_{t,d}$$

TERM FREQUENCY × INVERSE DOC FREQ = TF-IDF WEIGHT

Highest Score

HIGH TF • HIGH IDF

Frequent in this doc, rare in others.
"Best descriptive terms"

Medium

HIGH TF • LOW IDF

Frequent in this doc, but common everywhere.
"Stop words, common topics"

Medium

LOW TF • HIGH IDF

Rare term, but only appears once here.
"Passing mention"

Lowest

LOW TF • LOW IDF

Rarely in this doc, common everywhere.
"Noise"

TF-IDF Calculation Example

CORPUS (N=3)

D1: "car insurance"

D2: "auto insurance"

D3: "best car"

Term: "car"

TF (in D1)	1
DF (D1, D3)	2
IDF = log(3/2)	0.176

0.176

Low Weight (Common)

Term: "auto"

TF (in D2)	1
DF (D2 only)	1
IDF = log(3/1)	0.477

0.477

High Weight (Rare)

Vector Space Model (VSM)

HIGH-DIMENSIONAL SPACE

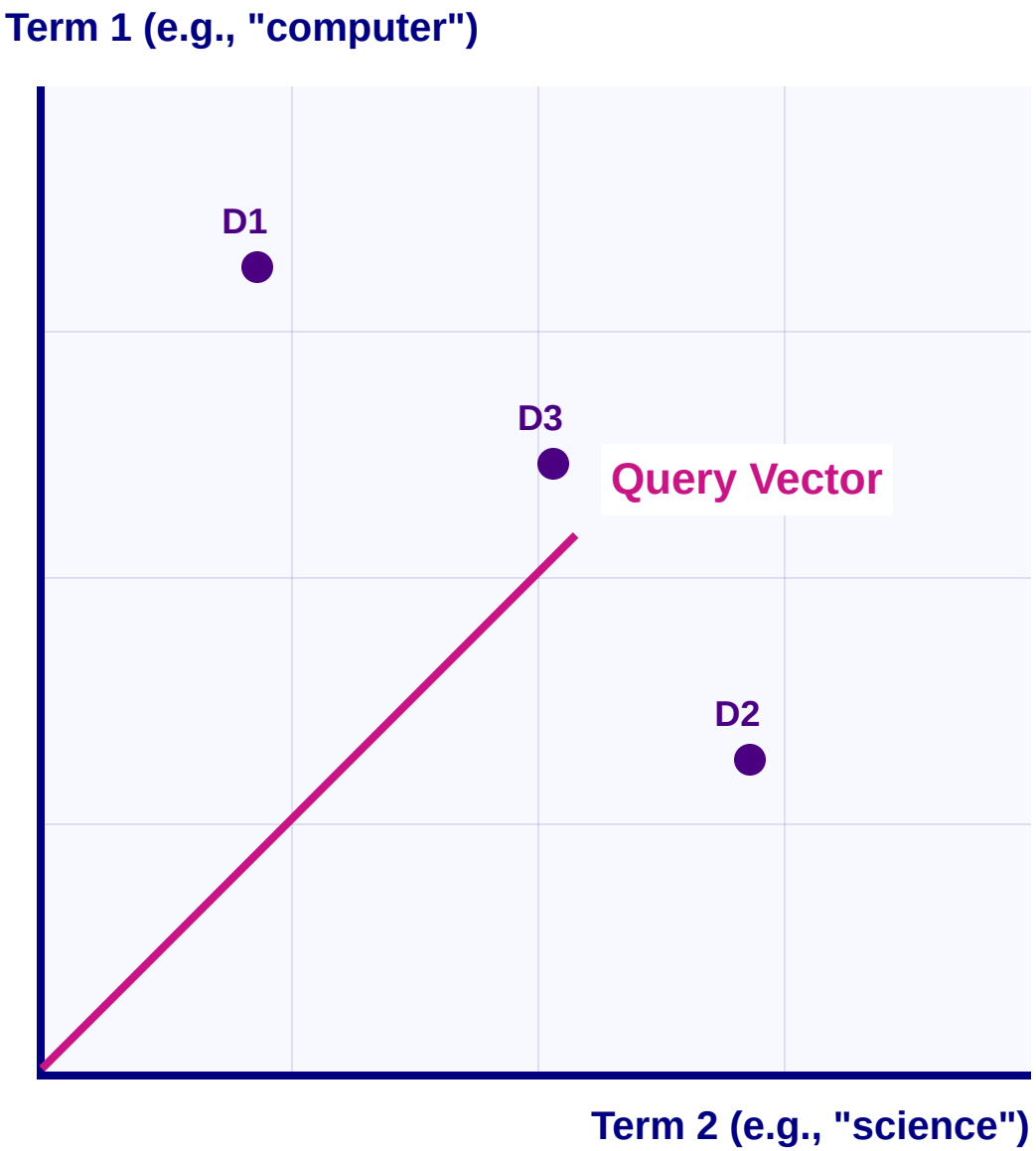
Each unique term in the vocabulary is a **dimension** (axis). A vocabulary of 10,000 terms = a 10,000-dimensional space.

DOCUMENTS AS POINTS

Every document is a **point** (or vector) in this space. Its coordinates are determined by the weights of its terms.

QUERIES AS VECTORS

A query is treated just like a short document. It is also a vector in the same space.



Documents as Vectors

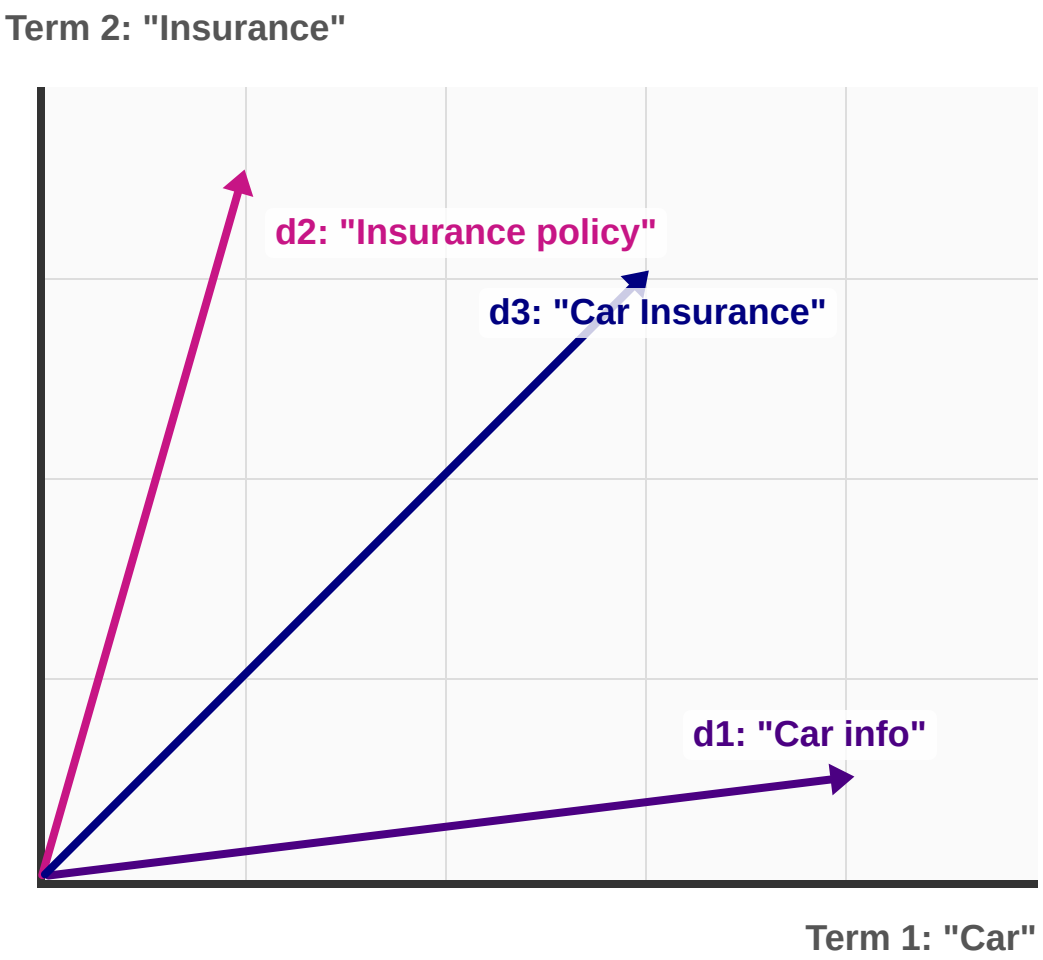
THE CONCEPT

We treat each document as a point (or vector) in a high-dimensional space.

DIMENSIONS

Each unique term in the vocabulary is a **dimension** (axis).
Vocabulary size V = 50,000 → 50,000 dimensions.

$$d_j = (w_{1,j}, w_{2,j}, \ldots, w_{V,j})$$



** Note: This space is extremely sparse. Most documents contain only a tiny fraction of the vocabulary.*

Euclidean Distance

$$d(q, d) = \sqrt{\Sigma(q_i - d_i)^2}$$

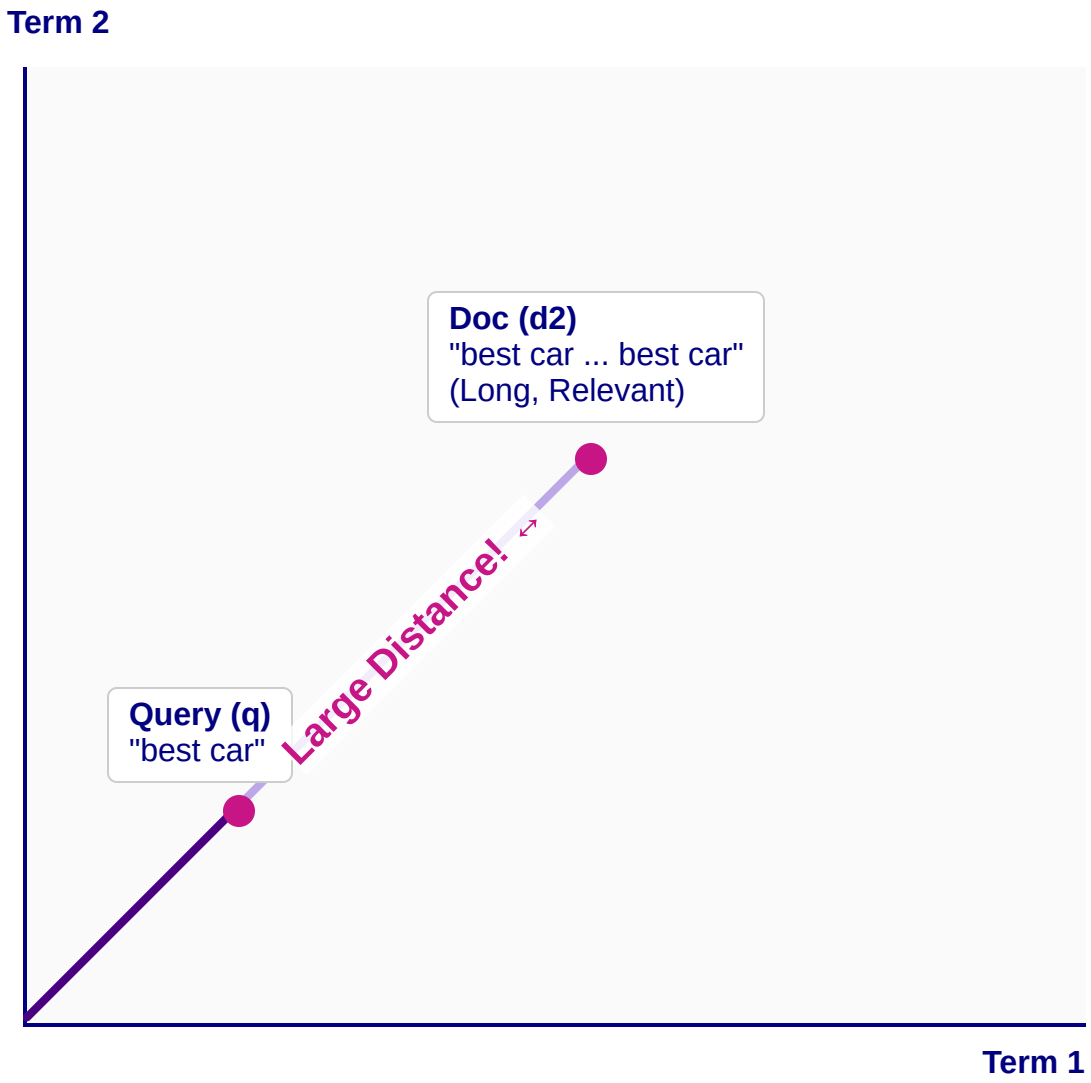
Measures the straight-line distance between points.

The Problem: Length Bias

Euclidean distance is sensitive to vector length.

A document that repeats the query terms many times (e.g., spam) will have a **very large distance** from the query, even though it is highly relevant.

Conclusion: We need a measure based on angle, not distance.

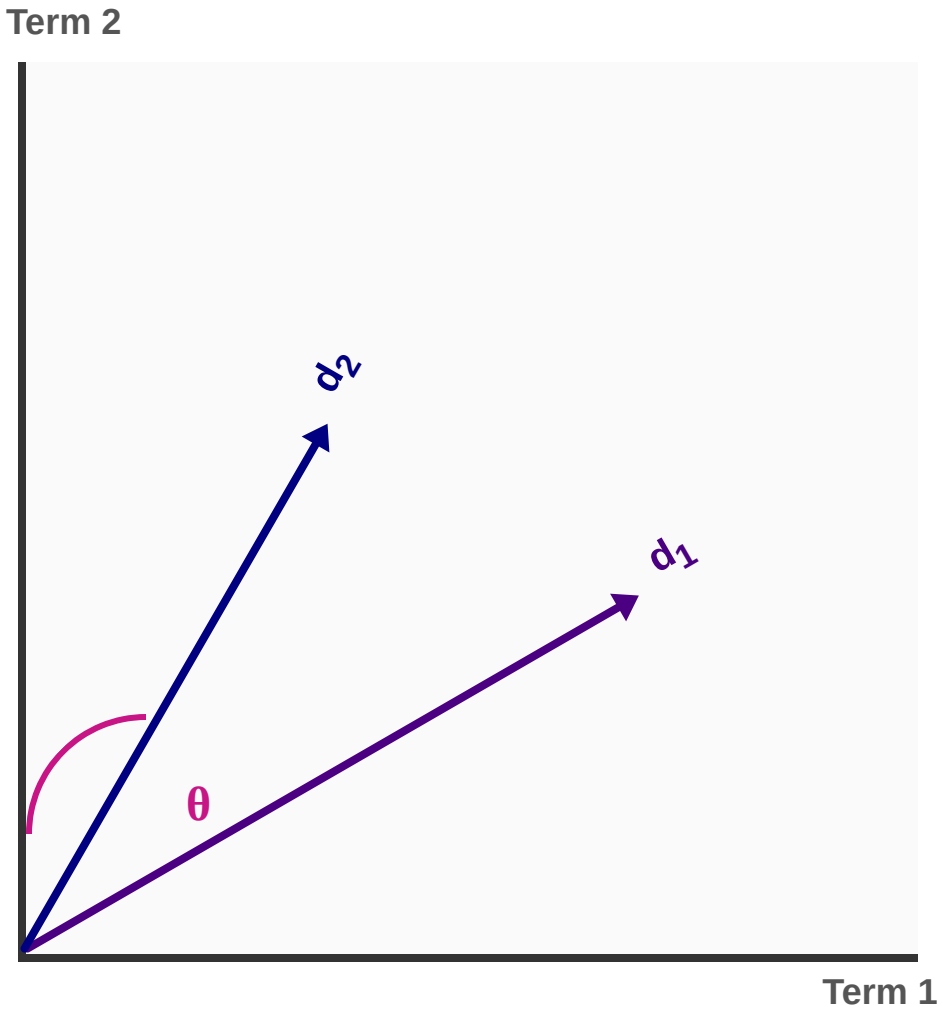


Cosine Similarity

THE CORE IDEA

Instead of distance, we measure similarity by the **cosine of the angle (θ)** between two vectors.

- **Angle = 0°** → Cosine = 1
  (Vectors are identical)
- **Angle = 90°** → Cosine = 0
  (Vectors are orthogonal/unrelated)
- **Magnitude is ignored**
  (Document length doesn't bias the score)



Cosine Similarity Formula

$$\mathrm{sim}(d_1, d_2) = \frac{V(d_1) \cdot V(d_2)}{|V(d_1)|\,|V(d_2)|}$$

DOT PRO

Measure
between
if they sh
terms.

EUCLIDE

Normali
long doc
dominate

=	↔	
1.0	**0.0**	**-1.0**
Identical (0°)	No Overlap (90°)	Opposite (180°)
		(Not usually in IR)

Length Normalization

THE PROBLEM

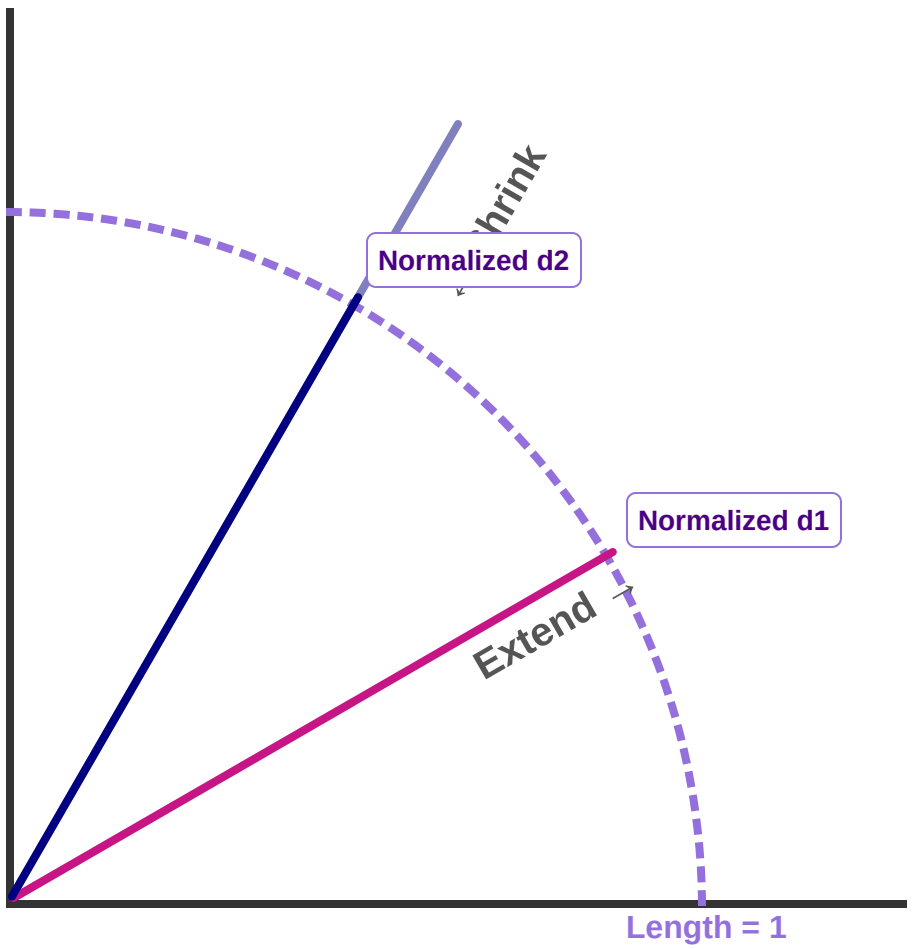
Long documents have higher term frequencies and more terms. This gives them larger vector magnitudes, unfairly boosting their scores.

THE SOLUTION

Force all vectors to have **Unit Length (Length = 1)**. This projects every document onto the surface of a hypersphere.

$$V_{norm} = V / |V|$$

Divide each component by the Euclidean length.



Cosine Calculation: Step 1

Step 1: Raw Term Frequencies (tf)

QUERY (Q)

`"gold silver truck"`

Doc 1 (d1)

`"Shipment of gold damaged in a fire."`

Doc 2 (d2)

`"Delivery of silver arrived in a silver truck."`

Doc 3 (d3)

`"Shipment of gold arrived in a truck."`

Term	Query	d1	d2	d3
gold	1	1	0	1
silver	1	0	2	0
truck	1	0	1	1
arrived	0	0	1	1
damaged	0	1	0	0
delivery	0	0	1	0
fire	0	1	0	0
shipment	0	1	0	1

** Stop words ("of", "in", "a") are removed.*

Step 2: TF-IDF Weighting

$W = TF \times IDF$ We multiply the raw frequency (TF) by the term's global importance (IDF).

Term	IDF	Query Vector		Doc 1 Vector		Doc 2 Vector	
		TF	W	TF	W	TF	W
"best"	1.5	1	1 × 1.5 **1.5**	0	0 × 1.5 **0**	1	1 × 1.5 **1.5**
"car"	0.3	1	1 × 0.3 **0.3**	1	1 × 0.3 **0.3**	1	1 × 0.3 **0.3**
"insurance"	1.2	1	1 × 1.2 **1.2**	1	1 × 1.2 **1.2**	0	0 × 1.2 **0**

Notice: The rare term "best" (1.5) contributes 5x more weight than "car" (0.3).

Cosine Calculation: Step 3 - Normalization

Goal: Divide each vector component by its length (magnitude) to get a **Unit Vector**.

EXAMPLE CALCULATION

Raw Vector V = [3, 4]

↓

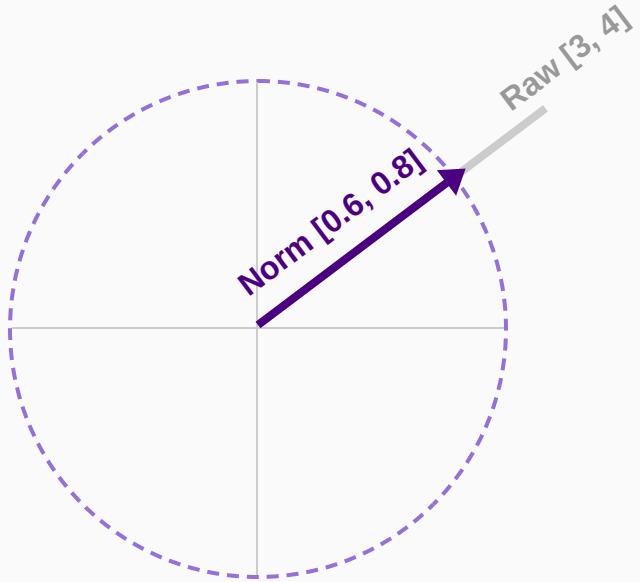
Length $|V| = \sqrt{(3^2 + 4^2)}$
$= \sqrt{(9 + 16)} = \sqrt{25}$
= 5

↓

Divide components by 5:

`[0.6, 0.8]`

(Note: $0.6^2 + 0.8^2 = 1$)



Unit Circle (Radius = 1)

Cosine Calculation: Step 4 (Dot Product)

Query (q)	0.707	0.707	0.000
	×	×	×
Doc (d)	0.500	0.800	0.300
	0.354	0.566	0.000
		+	+

COSINE SIMILARITY SCORE

0.920

Highly Relevant

Efficient Scoring

The Bottleneck

Computing exact cosine scores for **every document** containing query terms is too slow for large collections.

```
Query: "data mining"
Docs containing "data": 50M
Docs containing "mining": 10M
Operations: ~60 Million
```

The Strategy

We only need the **Top K** results (e.g., top 10). We can use heuristics to skip low-scoring documents.

- **Index Elimination**
- **Champion Lists**
- **Cluster Pruning**

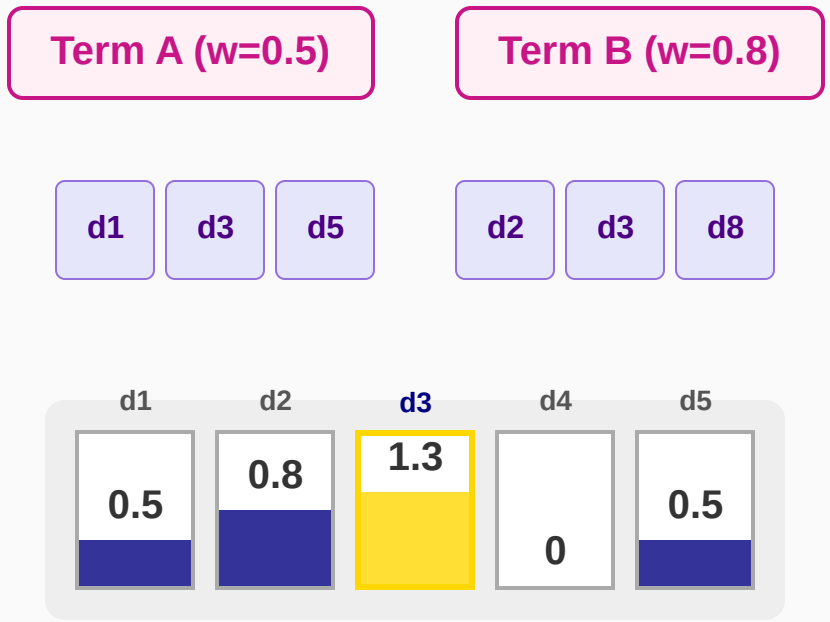
Fast Cosine Score Computation

THE INSIGHT

Instead of iterating through all N documents (which is slow), we iterate through the **query terms**. We only touch documents that actually contain at least one query term.

1. Initialize an empty array `Scores[]`.
2. For each query term t, fetch its Postings List.
3. For each doc d in the list, add the term's contribution to `Scores[d]`.
4. Return the top K documents from `Scores[]`.

ACCUMULATOR UPDATE PROCESS



d3 appears in both lists,
so it accumulates scores from both Term A and Term B.

Index Elimination

THE STRATEGY

Instead of scoring every document that contains *any* query term, we only consider documents that contain **discriminative terms** (High IDF).

THE LOGIC

Low-IDF terms (e.g., "the", "in") appear in almost all documents. They do not help rank the top results effectively.

THE BENEFIT

Drastically reduces the number of accumulators needed, speeding up query

Query Processing Example

`"the catcher in the rye"`

"the"	✗
"catcher"	✓
"in"	✗
"rye"	✓

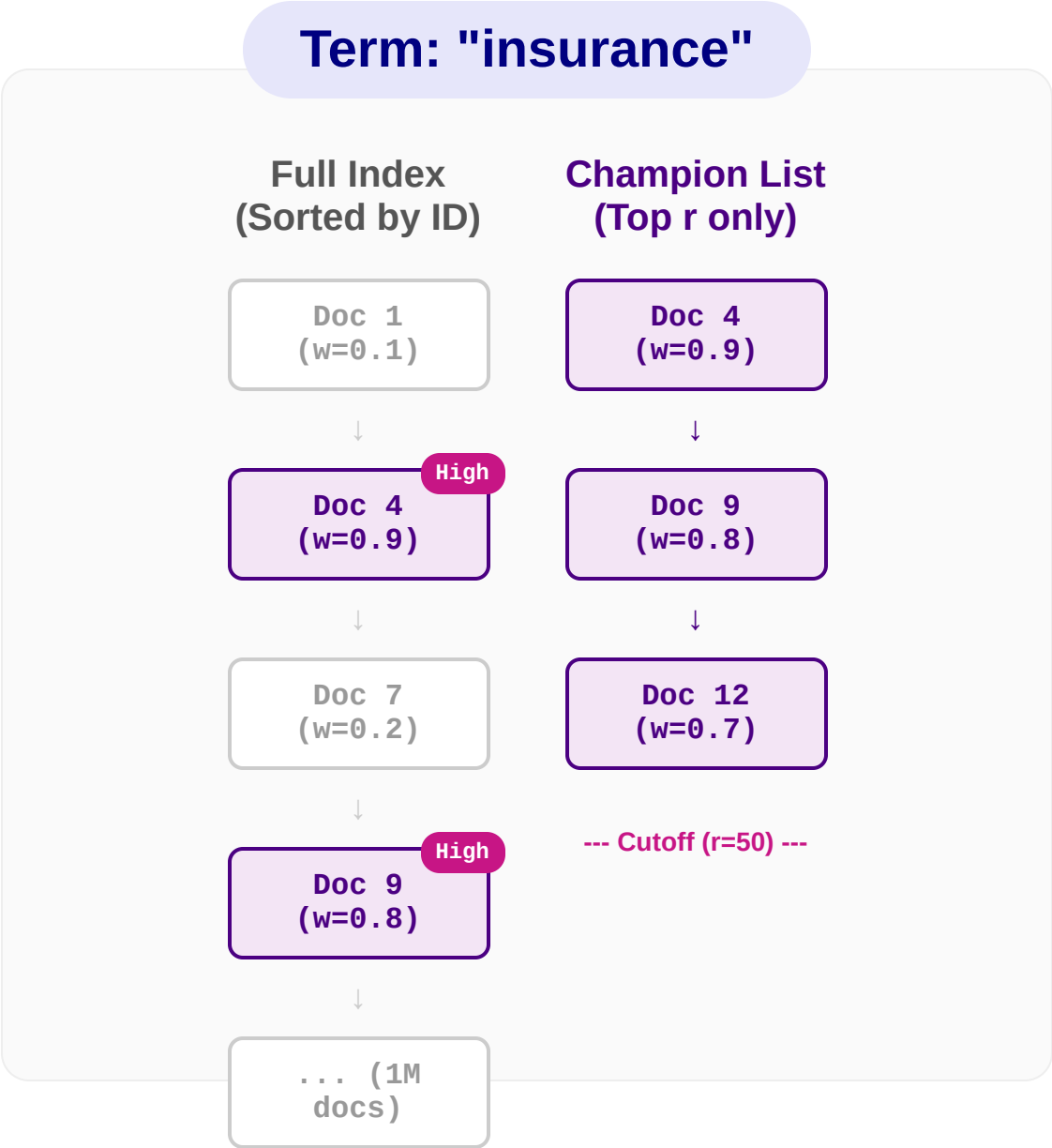
Only fetch postings for "catcher" & "rye"

Champion Lists (Tiered Indexes)

THE CORE IDEA

Pre-compute a list of the **top r** documents with the highest weights for each term. We call this the *Champion List*.

- **Query Time:** Only intersect the Champion Lists first.
- **Pruning:** Ignore documents with low TF-IDF scores that are unlikely to make the top K results.
- **Fallback:** If the result set is too small (< K), fall back to the full index.



Cluster Pruning

1. PRE-PROCESSING (OFFLINE)

Pick $\sqrt{N}$ documents as **Leaders**. Assign every other document (Follower) to its nearest Leader.

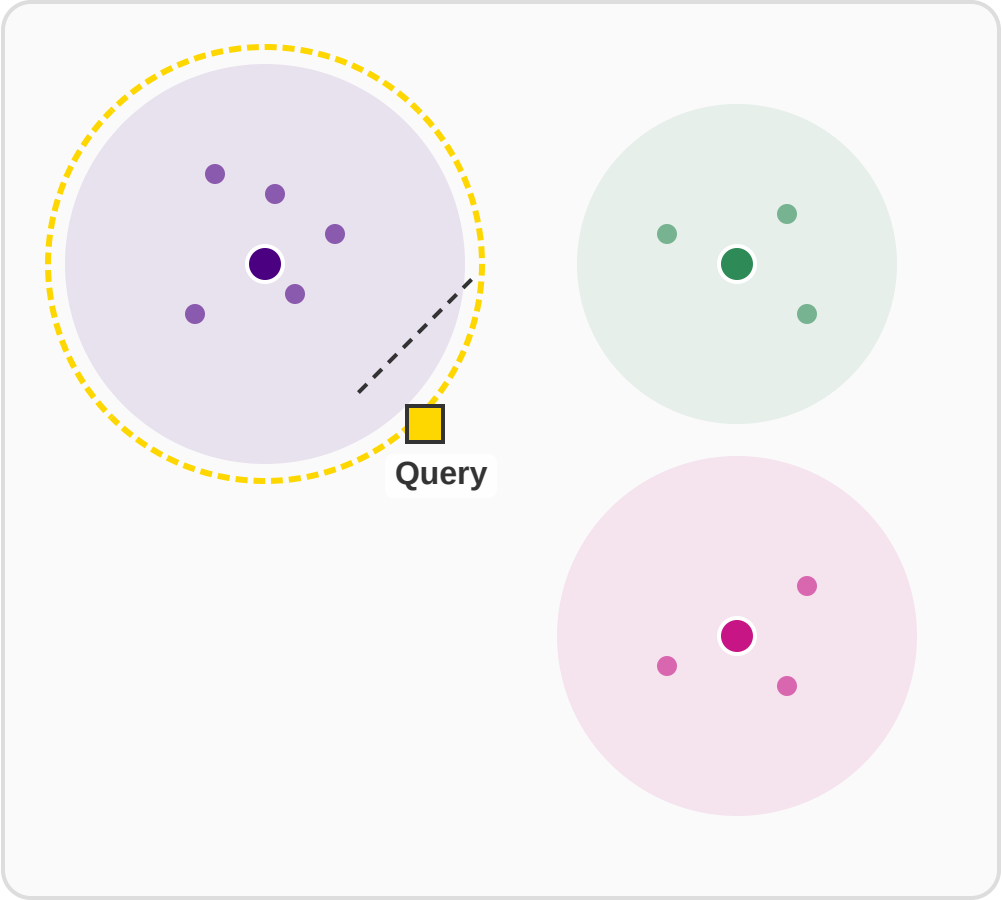
2. QUERY PROCESSING (ONLINE)

Compare query q to all Leaders. Identify the nearest Leader.

3. SEARCH

Compute cosine scores **only** for the Followers in that Leader's cluster. Ignore the rest.

Complexity: O(N) → O(√N)



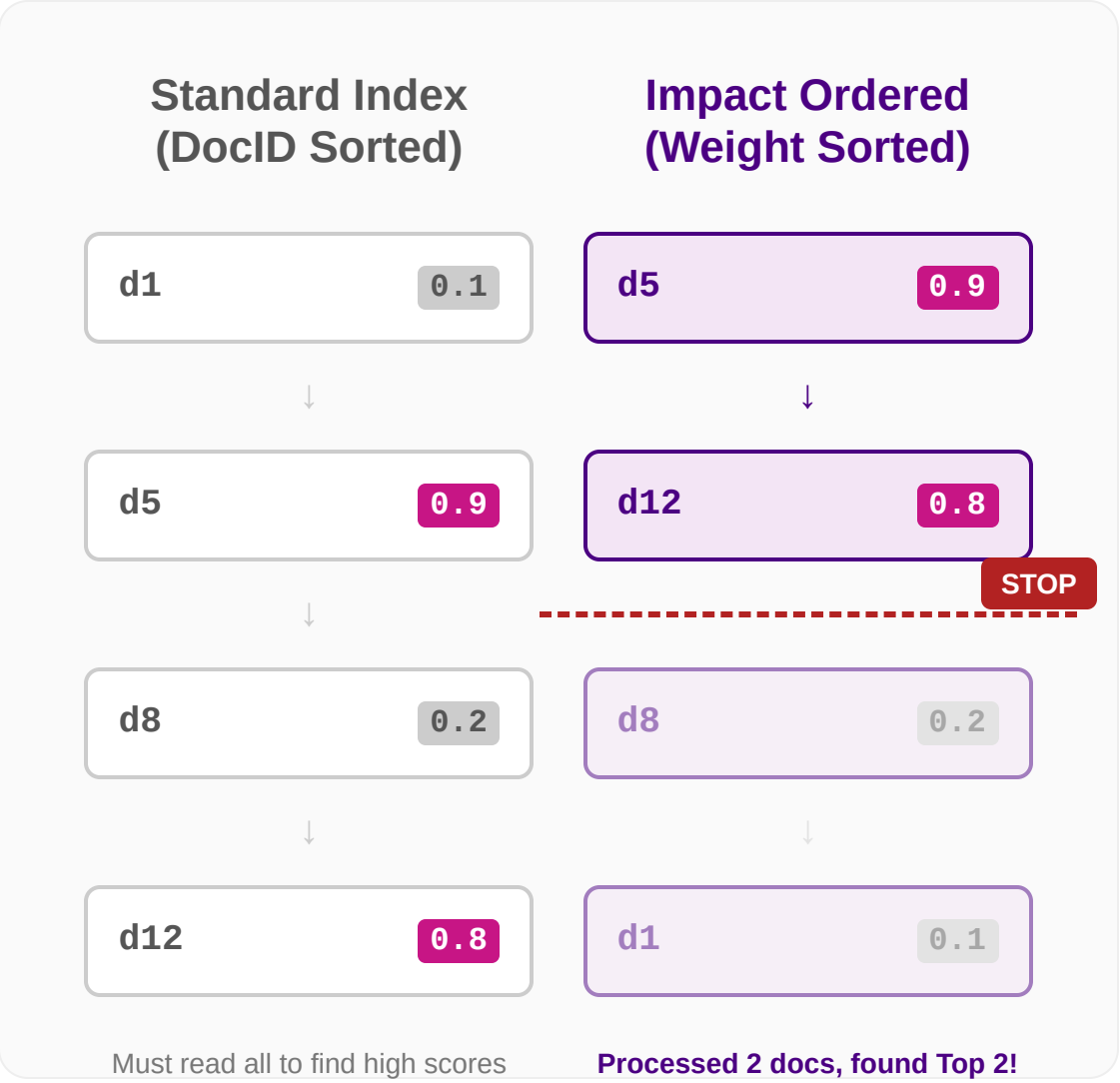
Leader Follower Query

Impact Ordering

THE CONCEPT

Instead of sorting postings by **DocID** (standard), sort them by **Weight** (TF-IDF score).

Feature	Standard	Impact Ordered
Sorting	By DocID (1, 2, 3...)	By Score (0.9, 0.8...)
Retrieval	Must scan full list	**Can stop early (Top K)**
Compression	**Excellent (Delta)**	**Poor (Random IDs)**



Modern Twist: Okapi BM25

The **Industry Standard** ranking function. Default in Elasticsearch, Lucene, and Solr.

FULL NAME

Best Matching 25 (Iteration #25)

ORIGIN

Developed in the 1990s at City University London (Okapi system).

CORE IDEA

A Probabilistic Information Retrieval model that improves upon TF-IDF by adding **tunable parameters** for term saturation and length normalization.

Why "Modern"?

It fixes the "linear growth" problem of raw TF and handles documents of varying lengths more

$$\text{Score}(D,Q) = \sum \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}$$

k_1

TERM SATURATION

Controls how quickly term frequency impact levels off. (Typical: 1.2 - 2.0)

b

LENGTH NORM

Controls penalty for long documents. (Typical: 0.75)

Term Saturation in BM25

THE PROBLEM WITH TF

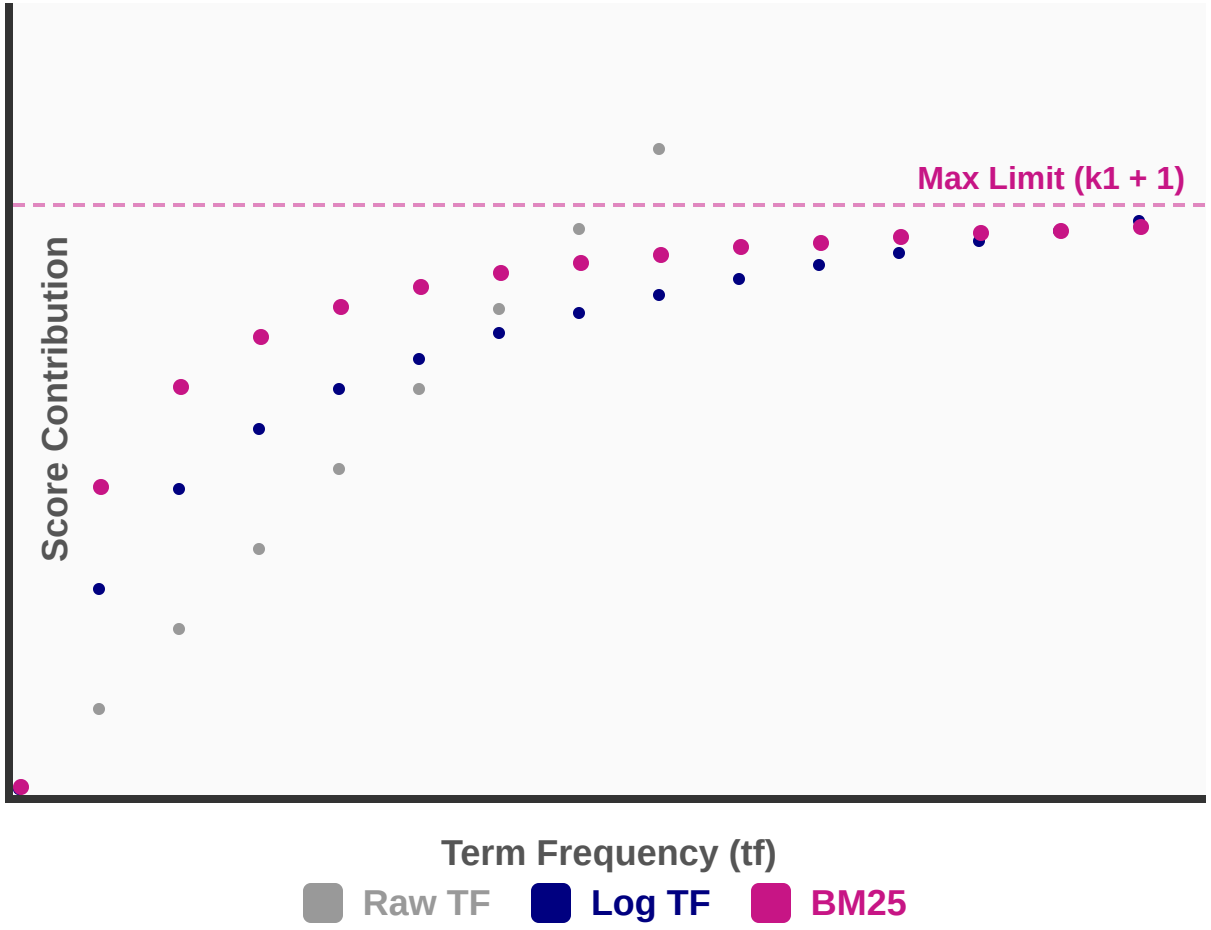
Raw TF grows linearly. A document with 100 occurrences of "cat" is not 100 times more relevant than one with 1 occurrence. Even Log TF grows indefinitely.

THE BM25 SOLUTION

Saturation: The score contribution of a term approaches a maximum limit ($k_1 + 1$) as $tf \to \infty$.

$$\text{Score} \propto (k_1 + 1)\, tf / (k_1 + tf)$$

k_1 controls saturation speed (typically 1.2 - 2.0)



Length Normalization in BM25

$$B = (1 - b) + b \cdot (dl / avgdl)$$

The normalization factor in the denominator.

The Parameter *b*

Controls the strength of length normalization.

- **b = 1:** Full normalization (like Cosine). Penalizes long docs heavily.
- **b = 0:** No normalization. Long docs are treated same as short ones.

Why is this better?

It allows tuning. We usually want *some* penalty for length, but not as harsh as cosine normalization.

Tuning Parameter *b*

b = 0.75

0 (None) **1 (Full)**

Standard Default

Verbose Document
Long because it repeats words? **Penalize it.**

Comprehensive Document
Long because it covers many topics? **Keep it.**

SMART Notation

DOCUMENT			·	QUERY		
d	d	d	·	q	q	q

1. TERM FREQUENCY	2. DOC FREQUENCY	3. NORMALIZATION
n natural: tf	**n** no: 1	**n** none: 1
l logarithm: 1+log(tf)	**t** idf: log(N/df)	**c** cosine: $1/\sqrt{(w_1^2+...)}$
a augmented: 0.5+0.5*...	**p** probabilistic	**u** pivoted unique
b boolean: 1 if tf>0		

Common Standard: `lnc.ltc`

Document: Log TF, No IDF, Cosine Normalization

Query: Log TF, IDF Weighting, Cosine Normalization

Visualizing the Vector Space

The Space

Imagine a high-dimensional space where each **term** is an axis (e.g., 10,000 dimensions).

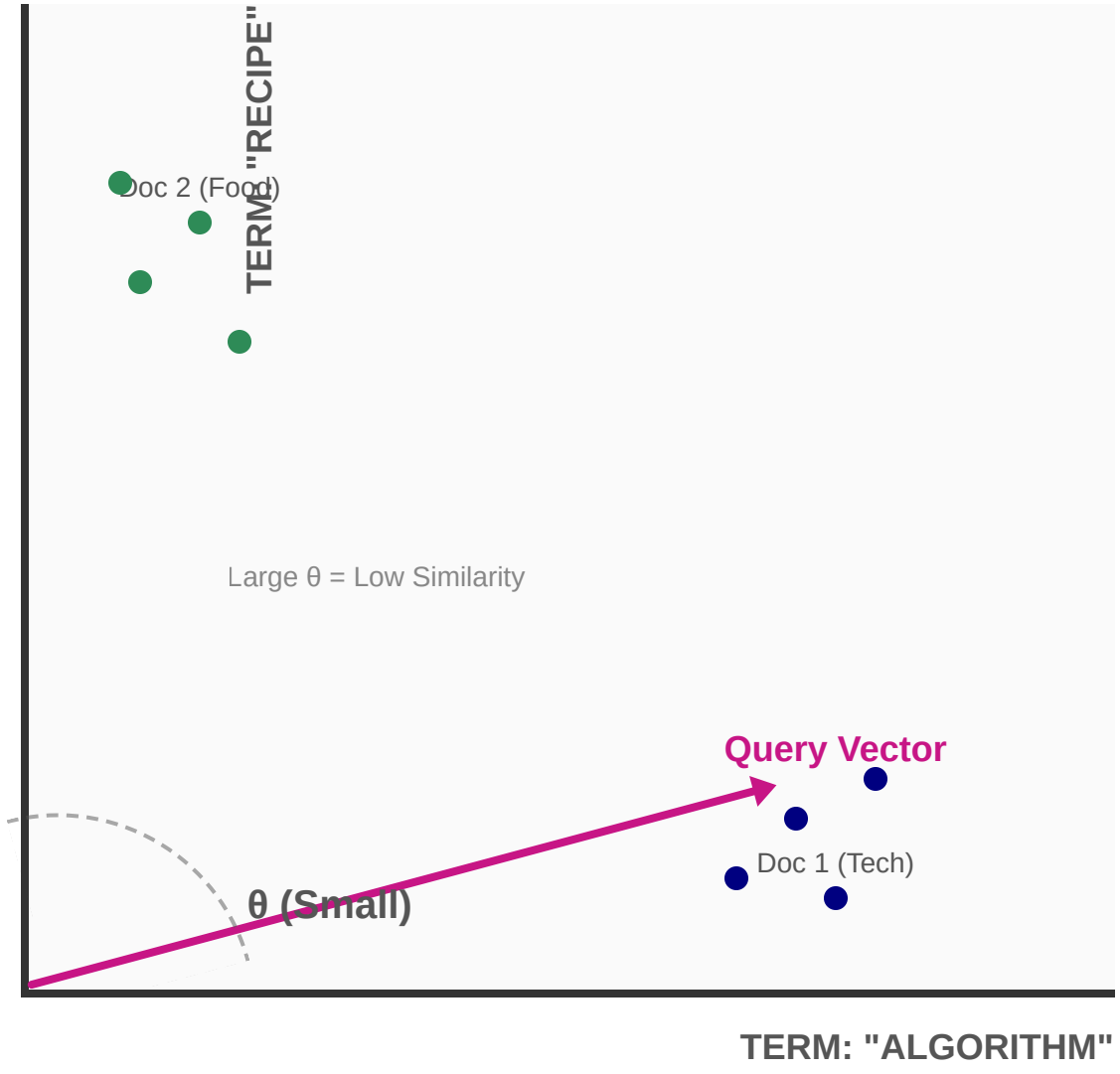
The Documents

Each document is a **point** (vector) in this space. Its coordinates are the TF-IDF weights.

The Query

The user's query is also a vector. We search for documents pointing in the **same direction**.

> *"Similarity is not about distance (Euclidean), but about **orientation** (Cosine Angle)."*



Limitations: The Semantic Gap

The Vector Space Model relies on **exact keyword matching**. It treats words as independent symbols and does not understand meaning.

= Synonymy

Different words that have the same meaning.

Problem: Low Recall

We miss relevant documents that use different words.

EXAMPLE

Query: "car" → Misses "automobile"

Query: "buy" → Misses "purchase"

🔀 Polysemy

One word that has multiple meanings.

Problem: Low Precision

We retrieve irrelevant documents with the wrong context.

EXAMPLE

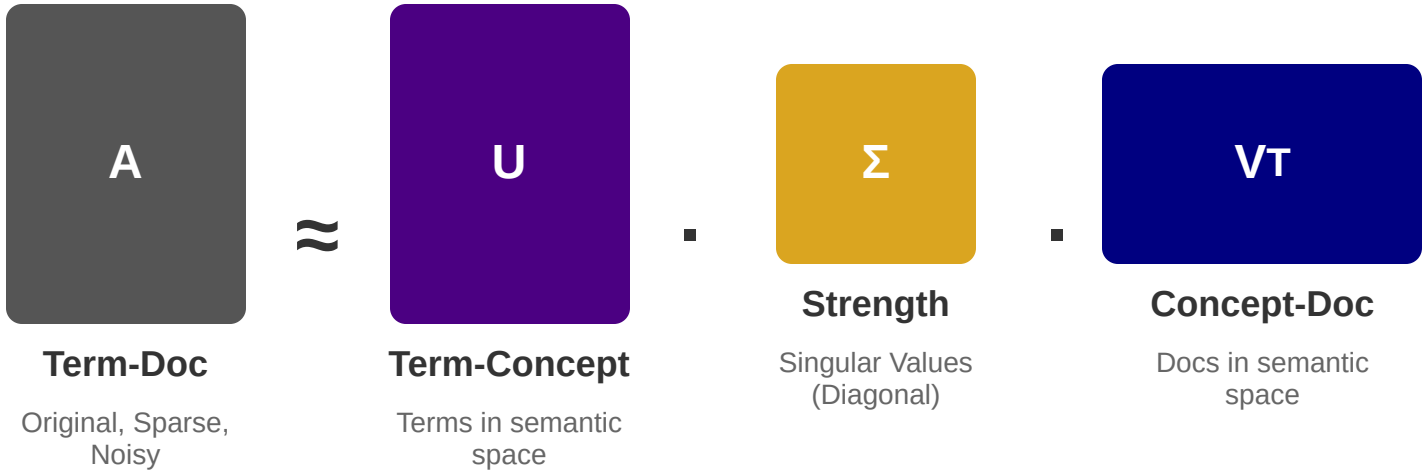
Query: "jaguar" → Retrieves "car" AND "animal"

Query: "apple" → Retrieves "fruit" AND "tech"

The Vocabulary Mismatch Problem: Users and authors often use different words to describe the same concept.

Latent Semantic Indexing (LSI)

The Vocabulary Mismatch Problem: Standard VSM treats "car" and "automobile" as orthogonal (unrelated) because they are different strings. We need to capture *meaning*.



Latent Concepts

Maps terms to a lower-dimensional "concept space" ($k \approx 100-300$). Synonyms are mapped close together.

Noise Reduction

By keeping only the top k singular values, we remove random noise and capture the underlying structure.

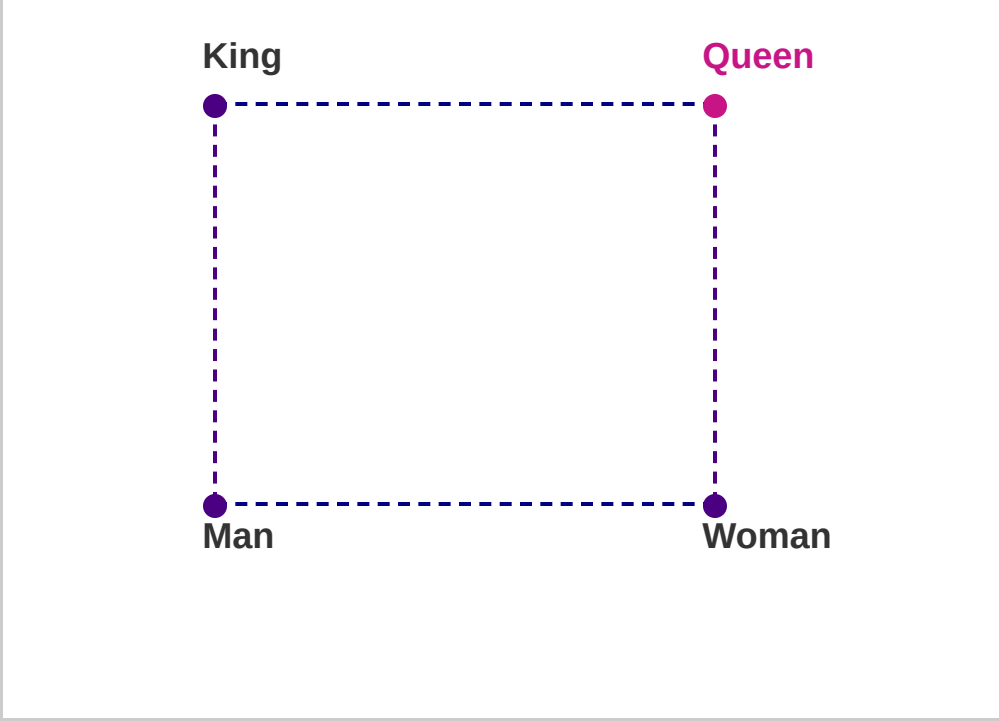
Word Embeddings (Word2Vec)

THE PROBLEM WITH VSM

TF-IDF vectors are **Sparse** (mostly zeros) and **High-Dimensional** (size of vocabulary). They fail to capture semantic meaning (e.g., "car" and "automobile" are orthogonal).

Feature	TF-IDF (Sparse)	Embedding (Dense)
Dimensions	100,000+	50 - 300
Values	Discrete counts	Continuous reals
Semantics	None (Exact match)	**Rich meaning**

Key Idea: "You shall know a word by the company it keeps." (Firth, 1957).



`King - Man + Woman ≈ Queen`

Vector Arithmetic captures semantic relationships.

Sparse vs. Dense Vectors

Sparse Vectors

```
[0, 1.4, 0, 0, 0, 2.1, 0, ... 0]
Length: 100,000+ (Vocabulary Size)
```

High Dimensionality

One dimension per word in vocabulary.

Exact Matching

Great for specific keywords (e.g., product codes).

Interpretable

We know exactly what dimension 452 means ("cat").

Dense Vectors

```
[0.12, -0.45, 0.88, 0.03, -0.19]
Length: 300 - 1024 (Fixed Size)
```

Low Dimensionality

Compressed representation of meaning.

Semantic Matching

Captures synonyms and context (e.g., "car" ≈ "auto").

"Black Box"

Dimensions are abstract features, hard to interpret.

Modern Search uses Hybrid Retrieval: Sparse (Keywords) + Dense (Semantics)

Case Study: E-commerce Search

"red running shoes"

THE CHALLENGE

A "Bag of Words" model might rank a "Red T-shirt" highly just because it mentions "red" many times. We need **structure**.

FIELD-BASED SCORING

We split the document vector into fields and assign different weights (w_f).

Title	w = 1.0
Category	w = 0.8
Description	w = 0.5

Nike Air Zoom - Red

SHOES / RUNNING

High performance running gear. Lightweight mesh...

$120.00

Relevance Score Calculation

Title Match ("Red")	1.0 × 0.9 = 0.90
Category Match ("Shoes")	0.8 × 0.8 = 0.64
Desc Match ("Running")	0.5 × 0.4 = 0.20
Total Score	**1.74**

Case Study: News Recommendation

Content-Based Filtering

We recommend articles that are **semantically similar** to what the user has read before, using Vector Space Model similarity.

1 User History

User reads articles about "Space Exploration".

Mars Rover Lands
Terms: Mars, NASA, Space

2 Build Profile

Create a **User Vector** (Centroid) by averaging read article vectors.

V_{user} = [0.8, 0.9, 0.1...]

3 Compute Similarity

Compare V_{user} with vectors of all **new articles** using Cosine Similarity.

$Sim(V_{user}, V_{new})$

4 Recommend

Show top-ranked articles.

SpaceX Launch
Score: 0.95

Lab: TF-IDF in Python

```python
from sklearn.feature_extraction.text import TfidfVectorizer

# 1. Define Corpus
corpus = [
    'the cat sat on the mat',
    'the dog sat on the log',
    'cats and dogs are enemies'
]

# 2. Initialize & Transform
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(corpus)

# 3. Inspect
print(vectorizer.get_feature_names_out())
print(X.shape)
```

1. **Import Library**
   Use `scikit-learn`, the industry standard for ML in Python.

2. **Fit & Transform**
   Learns the vocabulary AND converts documents to TF-IDF vectors in one step.

3. **Sparse Matrix**
   The output X is a sparse matrix (only stores non-zero values) to save memory.

```
Output:
['and' 'are' 'cat' 'cats' 'dog' ...]
(3, 10)
```

Lab: Cosine Similarity in Python

1. **Import Library**
   Use `sklearn.metrics.pairwise`. It's optimized for sparse matrices.

2. **Prepare Vectors**
   Transform your query and documents into TF-IDF vectors using the same vectorizer.

3. **Compute Similarity**
   Calculate the dot product. Since TF-IDF vectors are usually normalized, this equals the Cosine Similarity.

```
from sklearn.metrics.pairwise import cosine_similarity # 1. Define Query and Document query = "search engine algorithm" doc = "algorithms for search engines are complex" # 2. Transform to Vectors tfidf_matrix = vectorizer.fit_transform([query, doc]) # 3. Compute Cosine Similarity # Returns a matrix (Query vs Doc) similarity = cosine_similarity(tfidf_matrix[0], tfidf_matrix[1]) print(f"Similarity Score: {similarity[0][0]:.4f}")
```

Efficiency Tip

`linear_kernel` is faster than `cosine_similarity` if your data is already normalized (which TF-IDF usually is).

OUTPUT TERMINAL

```
> Similarity Score: 0.6824
```

Summary: From Boolean to Vectors

Boolean Retrieval

- Exact match model (AND, OR, NOT).
- Good for expert users and legal search.
- **Limitation:** No ranking, "Feast or Famine".

Term Weighting (TF-IDF)

- **TF:** Frequency within document.
- **IDF:** Rarity across collection.
- Balances local relevance with global specificity.

Vector Space Model

- Docs & Queries = Vectors in high-dim space.
- **Cosine Similarity:** Measures angle (orientation).
- Allows partial matching and ranked results.

Efficient Scoring

- Exact Top-K is expensive ($O(N)$).
- **Heuristics:** Index Elimination, Champion Lists.
- Trade-off: Speed vs. Precision (Safe vs. Unsafe).

→ **Next Week: Web Crawling & Link Analysis**

Further Reading

PRIMARY TEXT

Introduction to Information Retrieval

Manning, Raghavan, & Schütze

Read Chapter 6

Scoring, Term Weighting, and the Vector Space Model.

Classic Paper

The foundational work on TF-IDF variants and normalization.

Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval.

Modern Overview

A concise history of VSM and probabilistic models.

Singhal, A. (2001). Modern Information Retrieval: A Brief Overview.

Online Resources

Stanford CS276 Lecture Notes (Lecture 6 & 7).

nlp.stanford.edu/IR-book/

Review Questions

Conceptual

QUESTION 1

Explain the "Bag of Words" assumption. What important linguistic information is lost when using this model?

QUESTION 2

Why do we use Log Frequency weighting ($1 + \log(tf)$) instead of raw term frequency? Give an example where raw count fails.

QUESTION 3

Compare Jaccard Coefficient vs. Cosine Similarity. Why is Cosine Similarity preferred for length-normalized retrieval?

Calculation

PROBLEM 1: TF-IDF

Given a collection size $N=1000$, document frequency $df_t=10$, and term frequency $tf_{t,d}=100$. Calculate the TF-IDF weight using base-10 log.

PROBLEM 2: VECTOR SPACE

Given two vectors: $A = (1, 1, 0)$ $B = (1, 0, 1)$

Compute the Cosine Similarity between A and B.

PROBLEM 3: HEURISTICS

If we use "Index Elimination" and only consider documents containing terms with high IDF, how might this affect **Recall**?